

Day4. 계층형 아키텍처 구현(분할·연결 구체화)

1. 수업내용 정리

계층을 나누는 이유: 한 계층의 변경이 다른 계층에 영향을 안주기 위해.

계층형 아키텍처 - 3계층 책임 분리

항목	Presentation Layer	Business Layer	Persistence Layer
Annotation	@RestController	@Service	@Repository/JPA
Method	HTTP요청/응답, JSON직렬화, 입력검증	비즈니스 규칙, 트랜잭션, 여러 Repo 조율	DB CRUD, 쿼리 실행, Entity 매핑
비고	비즈니스 로직 없음	HTTP/DB 개념 없음	비즈니스 로직 없음

단방향 의존 규칙: Controller -> Service -> Repository(역방향 절대 금지/계층 건너뛰기 금지)

1.1. 계층 경계의 데이터 설계

같은 데이터인데 계층별 별도 객체로 존재. 계층 경계를 넘을 때 변환이 필요.

Entity

- DB 테이블과 1:1 매핑 / JPA가 관리하는 영속 객체
- 외부에 직접 노출 금지
- 예: Post(id, title, content, createdAt)

DTO: Data Transfer Object

- 계층 간 데이터 전달 전용, Request/Response 분리
- 계층 경계마다 변환: Entity -> Response, DTO/RequestDTO -> Entity
- 예: PostRequest(title, content) / PostResponse(id, title)

VO: Value Object

- 불변값 객체, 동등성은 값으로 판단
- 실무에서는 DTO와 혼용되기도 함 - 개념 이해가 중요
- 예: Email("alice@example.com") - 유효성 포함

Entity를 Controller가 직접 반환하면? DB 구조 노출 + 순환 참조 + 불필요한 필드 노출

!! 반드시 DTO로 변환해서 반환

1.2. 데이터 접근을 인터페이스로 분리

Service가 구현체가 아닌 인터페이스로 Repository를 의존하는 이유

```
public interface PostRepository {
    Post save(Post post);
    Optional<Post> findById(Long id);
    List<Post> findAll();
    List<Post> findByAuthorId(Long authorId);
}

public class JdbcPostRepository implements PostRepository {
    // 메서드명만으로 자동 생성되지 않음- SQL을 직접 작성
    public List<Post> findByAuthorId(Long authorId) {
        String sql = "SELECT * FROM posts WHERE author_id = ?";
        ... // PreparedStatement로 직접 조회 + 매핑
    }
}
```

1.3. 단방향 호출 규칙과 비즈니스 책임

1.3.1. 올바른 단방향 흐름

1.3.2. 금지 패턴

1. Controller -> Repository 직접 호출: 비즈니스 로직이 Controller에 섞임, Service가 테스트 불가능
2. Service -> Controller 역방향 호출: 순환 의존 발생, HTTP 개념이 Service에 침투
3. Repository 안에 비즈니스 로직: DB변경 시 비즈니스 로직도 함께 수정

Service의 핵심: 비즈니스 규칙 + 트랜잭션 + Repository 호출 조율 -> HTTP와 DB는 몰라도 됨.

1.4. JPA 기초

1.4.1. ORM(Object-Relational Mapping)

- Java 객체 <-> DB 테이블 자동 매핑
- SQL 없이 JAVA 메서드로 DB조작

1.4.2. @Entity

- DB 테이블과 매핑되는 클래스
- @ID로 PK지정, @Column으로 컬럼 매핑

1.4.3. 영속성 컨텍스트

- JPA가 Entity를 관리하는 1차 캐시
- 변경 감지(Dirty checking)로 자동 UPDATE

2. 코드 분석

2.1. Post Domain 추가

파일 트리뷰

```
loginauth
├── LoginAuthApplication.java
├── auth
│   ├── domain
│   │   └── User.java
│   ├── dto
│   │   ├── LoginRequest.java
│   │   ├── SignUpRequest.java
│   │   └── AuthCheckResponse.java
│   ├── exception
│   │   ├── DuplicateUsernameException.java
│   │   └── InvalidCredentialsException.java
│   ├── repository
│   │   └── UserRepository.java
│   ├── service
│   │   └── AuthService.java
│   └── web
│       ├── AuthController.java
│       └── AuthExceptionHandler.java
├── post
│   ├── domain
│   ├── dto
│   ├── exception
│   ├── repository
│   ├── service
│   └── web
└── global
    ├── config
    │   └── SecurityConfig.java
    └── exception
```

```

├── GlobalExceptionHandler.java
├── security
│   ├── CookieService.java
│   ├── JwtAuthenticationFilter.java
│   ├── JwtProperties.java
│   └── JwtProvider.java

```

- 폴더 구조가 3개로 나뉘었음.
- 인증 관련: auth, 글쓰기 관련: post, 공용 처리 관련: global
- 이번 4차시에서는 post의 구조만 다룸.

2.1.1. Post.java

```

@Entity
@Table(name = "posts")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String content;

    @Column(nullable = false, length = 50)
    private String writer;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    protected Post() {
    }

    public Post(String title, String content, String writer) {
        this.title = title;
        this.content = content;
        this.writer = writer;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = this.createdAt;
    }

    public void update(String title, String content) {
        this.title = title;
        this.content = content;
        this.updatedAt = LocalDateTime.now();
    }

    public Long getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getContent() {
        return content;
    }

    public String getWriter() {

```

```

        return writer;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }
}

```

- @Entity: DB 테이블과 매칭되는 클래스
- @Table: DB 테이블의 이름
- @Id: PrimaryKey 매칭
- @Column: 테이블의 각 컬럼명
- 편의 메서드로 update()를 제공하여 각 데이터 값을 업데이트할 수 있음.

2.1.2. dto

```

// PostCreateRequest.java
public record PostCreateRequest(
    @NotBlank(message = "제목을 입력해주세요.")
    @Size(max = 200, message = "제목은 200자 이하여야 합니다.")
    String title,

    @NotBlank(message = "내용을 입력해주세요.")
    String content
) { }

// PostResponse.java
public record PostResponse(
    Long id,
    String title,
    String content,
    String writer,
    LocalDateTime createdAt,
    LocalDateTime updatedAt
) {
    public static PostResponse from(Post post) {
        return new PostResponse(
            post.getId(),
            post.getTitle(),
            post.getContent(),
            post.getWriter(),
            post.getCreatedAt(),
            post.getUpdatedAt()
        );
    }
}

// PostUpdateRequest.java
public record PostUpdateRequest(
    @NotBlank(message = "제목을 입력해주세요.")
    @Size(max = 200, message = "제목은 200자 이하여야 합니다.")
    String title,

    @NotBlank(message = "내용을 입력해주세요.")
    String content
) { }

```

- 계층 간 데이터 전달을 위한 클래스
- Response DTO는 from 정적 메서드를 통해 해당 post의 결과 값을 반환할 수 있도록 처리하고 있음.

2.1.3. repository

```
// PostRepository.java
public interface PostRepository extends JpaRepository<Post, Long> {

    List<Post> findAllByOrderByIdDesc();

}
```

- PostRepository는 JPA를 사용하기 때문에 상속을 기존 spring의 JpaRepository를 상속받음.
- JpaRepository<Post, Long>에서 Post는 관리할 Entity타입, Long은 PK타입을 의미.
- 기본으로 제공되는 db관련 메서드가 존재함.(save, findById, findAll, deleteById, existById, count)
- 다시 한 번 강조: Repository는 DB 접근만 담당하고, 작성자 검증이나 게시물 수정 가능 여부 같은 비즈니스 판단은 Service에서 처리

2.1.4. Service

```
// PostService.java
@Service
@Transactional(readOnly = true)
public class PostService {

    private final PostRepository postRepository;

    public PostService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    public List<PostResponse> findAll() {
        return postRepository.findAllByOrderByIdDesc()
            .stream()
            .map(PostResponse::from)
            .toList();
    }

    public PostResponse findById(Long postId) {
        return PostResponse.from(findPost(postId));
    }

    @Transactional
    public PostResponse create(PostCreateRequest request, String username) {
        Post post = new Post(request.title(), request.content(), username);
        return PostResponse.from(postRepository.save(post));
    }

    @Transactional
    public PostResponse update(
        Long postId,
        PostUpdateRequest request,
        String username
    ) {
        Post post = findPost(postId);
        verifyWriter(post, username);
        post.update(request.title(), request.content());
        return PostResponse.from(post);
    }

    @Transactional
    public void delete(Long postId, String username) {
        Post post = findPost(postId);
        verifyWriter(post, username);
        postRepository.delete(post);
    }

    private Post findPost(Long postId) {
        return postRepository.findById(postId)
            .orElseThrow(() -> new PostNotFoundException(postId));
    }
}
```

```

    }

    private void verifyWriter(Post post, String username) {
        if (!post.getWriter().equals(username)) {
            throw new PostAccessDeniedException();
        }
    }
}

```

- Service는 게시글 기능의 핵심 비즈니스 흐름을 담당.
- @Transactional(readOnly=True): 클래스 전체에 기본적으로 읽기 전용 트랜잭션을 적용함. 여기서는 조회만 하므로 DB변경이 없어 적절한 구문.
- @Transactional: DB의 상태를 변경하는 메서드에는 붙이는 어노테이션, 메서드 실행 중 예외가 발생하면 변경 사항이 롤백됨.
- findPost()로 조회한 Post는 JPA의 영속성 컨텍스트가 관리하는 Entity.
- post.update()로 필드를 변경하면 트랜잭션 종료 시점에 Dirty Checking을 통해 UPDATE 쿼리가 자동 실행됨.

2.1.5. Controller

```

// PostController.java
@RestController
@RequestMapping("/api/posts")
public class PostController {

    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @GetMapping
    public List<PostResponse> findAll() {
        return postService.findAll();
    }

    @GetMapping("/{postId}")
    public PostResponse findById(@PathVariable Long postId) {
        return postService.findById(postId);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public PostResponse create(
        @Valid @RequestBody PostCreateRequest request,
        Authentication authentication
    ) {
        return postService.create(request, authentication.getName());
    }

    @PutMapping("/{postId}")
    public PostResponse update(
        @PathVariable Long postId,
        @Valid @RequestBody PostUpdateRequest request,
        Authentication authentication
    ) {
        return postService.update(postId, request, authentication.getName());
    }

    @DeleteMapping("/{postId}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(
        @PathVariable Long postId,
        Authentication authentication
    ) {
        postService.delete(postId, authentication.getName());
    }
}

```

- Controller는 HTTP 요청을 받아 Service로 전달하고, Service 결과를 HTTP 응답으로 반환하는 역할만 담당함.
- 게시물 생성, 수정, 삭제 가능 여부 같은 비즈니스 판단은 Controller에서 하지 않음.
- @RequestMapping: 기본 URL을 지정함.
- @GetMapping: GET 요청을 처리하며 @PathVariable을 통해 URL의 postID 값을 메서드 파라미터로 받음.
- @PostMapping: POST 요청을 처리함. @RequestBody로 JSON 요청 본문을 PostCreateRequest로 변환함.
- @Valid로 DTO에 선언된 @NotBlank, @Size 검증을 실행함.
- @ResponseStatus를 통해 성공 시 201 Created를 반환함.
- @DeleteMapping: 삭제 성공 시 응답 본문 없이 204 No Content 반환